

Proyecto Parte 1 — Movimiento de un Termion en un campo orbital

ISIS 2112 — Diseño de Algoritmos — Semestre 2026-20

0. Identificación

Nombre Apellido — 2026xxxxx — Sección X
Nombre Apellido — 2026xxxxx — Sección X
Nombre Apellido — 2026xxxxx — Sección X

1. Algoritmo de solución

1.1 Modelado como problema de ruta óptima

El campo se modela como un grafo dirigido y ponderado $G = (V, E)$. Cada posición del campo es un vértice (i, y) , con i el número de órbita y y la posición dentro de ella, de modo que $|V| = n \cdot m$. Hay dos clases de arcos:

- **Movimientos internos:** entre (i, y) y $(i, y \pm 1)$, en ambos sentidos y con peso e_i , fijo dentro de cada órbita.
- **Portales:** un arco dirigido de (x_s, y_s) a (x_e, y_e) con peso 0.

El problema pedido es entonces el de la **ruta de costo mínimo** entre el vértice de entrada $(1, 1)$ y el de salida (n, m) ; si ese vértice es inalcanzable la respuesta es NO EXISTE. Como todos los pesos son no negativos, el algoritmo de Dijkstra resolvería el problema en $O((|V| + |E|) \log |V|)$. Sin embargo, la estructura particular de este grafo permite eliminar por completo el factor logarítmico.

1.2 Observación clave: el grafo de órbitas es un DAG

El enunciado garantiza que todo portal cumple $x_s < x_e$. Si se contrae cada órbita a un único supernodo, el grafo resultante solo tiene arcos que van de una órbita a otra de índice estrictamente mayor: es un **grafo dirigido acíclico** cuyo orden topológico ya viene dado, y es simplemente $1, 2, \dots, n$. En consecuencia, un termion *nunca* puede regresar a una órbita que ya abandonó, y los costos mínimos de la órbita i quedan completamente determinados por los de las órbitas $1, \dots, i-1$. Esto habilita una solución por **programación dinámica** que relaja los arcos en orden topológico en una sola pasada, sin cola de prioridad.

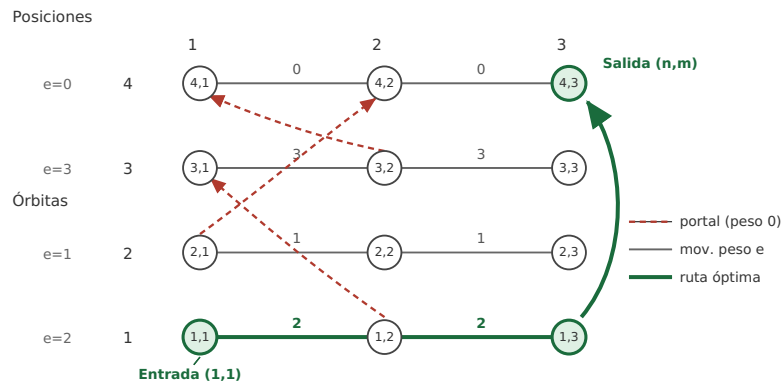


Figura 1. Campo del ejemplo visto como grafo ($n=4, m=3$). Los portales, en rojo punteado, solo apuntan hacia órbitas superiores: el grafo de órbitas es acíclico. En verde, la ruta óptima de costo 4.

1.3 Recurrencia de programación dinámica

Sea $C[i][y]$ la energía mínima necesaria para llegar desde la entrada hasta la posición (i, y) . El cálculo de la órbita i se hace en dos etapas.

(a) Costos de entrada a la órbita. Un termion llega a la órbita i solo por la entrada del campo o por algún portal proveniente de una órbita anterior:

$$\text{Ent}[i][y] = \min (\quad 0 \text{ si } (i,y) = (1,1) \quad ; \quad \min \{ C[x_s][y_s] : \text{portal } (x_s, y_s) \rightarrow (i,y) \} \quad) , \quad \infty \text{ si no hay ninguno.}$$

(b) Propagación dentro de la órbita. Una vez dentro, moverse de y' a y cuesta $|y - y'| \cdot e_i$, porque el camino más barato entre dos posiciones de la misma órbita es monótono (devolverse solo agrega pasos y $e_i \geq 0$). Por lo tanto:

$$C[i][y] = \min_{1 \leq y' \leq m} (\text{Ent}[i][y'] + |y - y'| \cdot e_i)$$

Evaluar esa expresión probando todos los pares costaría $O(m^2)$ por órbita. En cambio se calcula en $O(m)$ con **dos barridos lineales** sobre el arreglo, que no son otra cosa que la relajación de los arcos internos en los dos únicos órdenes topológicos posibles (hacia la derecha y hacia la izquierda):

$C \leftarrow \text{Ent}$
para $y = 2 \dots m$: $C[y] \leftarrow \min(C[y], C[y-1] + e_i)$ (cubre los $y' < y$)
para $y = m-1 \dots 1$: $C[y] \leftarrow \min(C[y], C[y+1] + e_i)$ (cubre los $y' > y$)

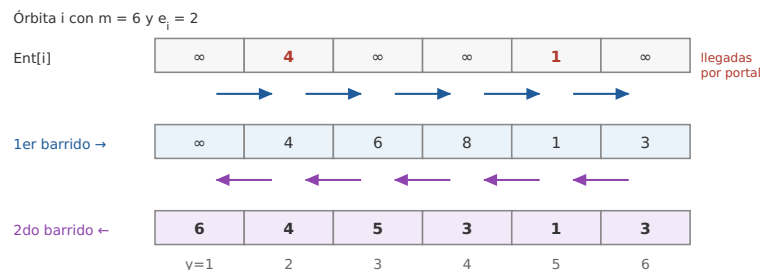


Figura 2. Cálculo de una órbita en tiempo $O(m)$. El primer barrido propaga los costos hacia la derecha y el segundo hacia la izquierda; la tercera fila contiene ya los valores $C[i][y]$ definitivos.

1.4 Algoritmo completo

Para cada caso de prueba se recorren las órbitas $i = 1, \dots, n$ en ese orden (el orden topológico del DAG). Para cada una: se inicializa el arreglo C con los costos de entrada, se aplican los dos barridos y, finalmente, por cada portal que sale de la órbita i se registra el costo $C[y_s]$ en la lista de llegadas de la órbita destino x_e , que aún no ha sido procesada. Al llegar a la órbita n , la respuesta es $C[m]$, o NO EXISTE si ese valor sigue siendo infinito.

Como cada órbita se procesa una sola vez y nunca se vuelve atrás, basta mantener **un solo arreglo de tamaño m** (la órbita actual) en lugar de la matriz completa de $n \cdot m$ costos. Las listas de portales entrantes y salientes se implementan como listas enlazadas sobre arreglos de enteros (*cabeza / siguiente*), lo que evita el sobre costo de estructuras dinámicas. La lectura se hace con un lector de bytes sobre la entrada estándar y la escritura se acumula en un único buffer, porque con hasta 10^6 portales por caso el costo de entrada/salida domina fácilmente el del cómputo.

2. Análisis de complejidades

2.1 Complejidad temporal

El trabajo por caso de prueba se descompone así:

Etapas	Costo	Justificación
Lectura de energías y portales	$O(n + p)$	Un número de operaciones constante por dato leído.
Inicializar y barrer cada órbita	$O(m)$ por órbita $\rightarrow O(n \cdot m)$	Tres recorridos lineales del arreglo de m posiciones.
Procesar portales	$O(p)$ en total	Cada portal se consulta exactamente dos veces: al salir y al llegar.

$$T(n, m, p) = O(n \cdot m + p)$$

Como el enunciado garantiza $p \leq n \cdot m$, el costo se simplifica a **$O(n \cdot m)$** , es decir, lineal en el tamaño del campo: a lo sumo 10^6 operaciones elementales por caso con los límites dados. La comparación con las alternativas naturales muestra la ganancia:

- Dijkstra con cola de prioridad sobre los $n \cdot m$ vértices: $O((n \cdot m + p) \log(n \cdot m))$, unas 20 veces más lento por el factor logarítmico, además del costo de mantener el montículo.
- Evaluar la recurrencia (b) directamente para todos los pares (y, y') : $O(n \cdot m^2) =$ hasta 10^9 operaciones, inviable.
- Bellman-Ford o relajación iterativa hasta estabilizar: $O(n \cdot m \cdot |E|)$, muchísimo peor.

El término $n \cdot m$ es inevitable en el peor caso porque cada órbita puede tener que propagar costos a lo largo de sus m posiciones. La cota es, además, ajustada: el algoritmo siempre ejecuta exactamente $3 \cdot n \cdot m$ pasos de barrido, sin dependencia del contenido de los datos.

2.2 Complejidad espacial

Estructura	Tamaño
Energías e_i y cabezas de las listas de portales	$O(n)$
Descripción de portales y sus costos de llegada	$O(p)$
Arreglo de costos de la órbita actual	$O(m)$

$$S(n, m, p) = O(n + m + p)$$

Lo importante de este resultado es que **no se almacena la matriz completa de $n \cdot m$ costos**, que en el peor caso ocuparía 10^6 valores de 64 bits (8 MB). Al recorrer las órbitas en orden topológico, los valores de una órbita ya procesada nunca vuelven a necesitarse salvo a través de los portales que salen de ella, y esos costos se guardan en un único arreglo de tamaño p .

Nota sobre el tipo de dato: el costo de una ruta puede alcanzar el orden de $n \cdot m \cdot \max(e_i) \approx 10^9$, valor que está cerca del límite de un entero de 32 bits con signo. Por eso los costos se acumulan en variables de 64 bits (long), y el valor “infinito” se representa como `Long.MAX_VALUE/4` para que las sumas intermedias nunca se desborden.

2.3 Verificación experimental

La implementación reproduce la salida del ejemplo del enunciado (4, 26, NO EXISTE) y fue contrastada contra una implementación de referencia con Dijkstra sobre el grafo completo en 800 campos generados al azar, con coincidencia total. En el peor caso permitido (5 casos de prueba con $n = m = 1000$ y $p = 10^6$ portales cada uno) la ejecución completa toma alrededor de 2 segundos, dominados por la lectura de los 75 MB de entrada.